FPT Education

FPT POLYTECHNIC

cdio™

Conceive Design Implement Operate



BÀI 6. THỰC HIỆN DỰ ÁN 4

Giảng viên:

https://caodang.fpt.edu.vn

MỤC TIÊU

- Xây dựng chức năng quản lý hoá đơn
- Xây dựng chức năng báo cáo thống kê
  - Thống kê sản phẩm bán chạy
  - Thống kê doanh thu



QUẢN LÝ HOÁ ĐƠN

- Chức năng quản lý hoá đơn cho phép hệ thống thực hiện các công việc sau:
  - Ghi nhận và theo dõi toàn bộ giao dịch bán hàng
  - Đảm bảo tính chính xác
  - Minh bạch và khả năng truy vết trong quá trình vận hành
- Nghiệp vụ chính:
  - Tạo và lưu hoá đơn bán hàng
  - Ghi nhận sản phẩm, số lượng, tổng tiền
  - Quản lý trạng thái hoá đơn (nháp/đã thanh toán/huỷ)

QUẢN LÝ HOÁ ĐƠN

Phân quyền sử dụng

- **Manager**:
  - Xem và quản lý toàn bộ hoá đơn.
  - Chuyển trạng thái từ đã thanh toán sang huỷ
- **Staff**:
  - Tạo hoá đơn nháp,
  - Cập nhật số lượng sản phẩm ở hoá đơn khi ở trạng thái nháp
  - Chuyển trạng thái hoá đơn
  - Xem danh sách hoá đơn của chính mình

BÁO CÁO VÀ THỐNG KÊ

- Chức năng báo cáo và thống kê cung cấp cái nhìn tổng quan về hiệu quả kinh doanh, hỗ trợ nhà quản lý đưa ra quyết định dựa trên dữ liệu
- Nghiệp vụ chính:
  - Thống kê sản phẩm bán chạy theo thời gian
  - Thống kê doanh thu theo ngày, tháng
  - Hiển thị dữ liệu dạng bảng và biểu đồ
  - Chỉ **Manager** được phép truy cập báo cáo



XÂY DỰNG CHỨC NĂNG QUẢN LÝ HOÁ ĐƠN

THÔNG TIN ĐƠN HÀNG

Cơ sở dữ liệu

dbo.bills		
int	id	PK
int	user_id	FK
varchar(10)	code	
date	created_at	
int	total	
varchar(10)	status	



Entity Class

```
public class BillDAL
{
    4 references
    public int Id { get; set; }
    3 references
    public int UserId { get; set; }
    3 references
    public string? Code { get; set; }
    3 references
    public DateTime CreatedAt { get; set; }
    3 references
    public int Total { get; set; }
    3 references
    public string? Status { get; set; }
```

Các phương thức cần CRUD ở ***BillDAL***

```
5 references
private static BillDAL FromDataRow(DataRow row)...

0 references
public static List<BillDAL> GetAll()...

0 references
public static BillDAL? GetById(int id)...

0 references
public static int Insert(BillDAL bill)...

0 references
public static bool Update(BillDAL bill)...

0 references
public static bool Delete(int id)...
```

Các phương thức ***Business Logic*** ở ***BillDAL***

```
0 references
public static BillDAL? GetByCode(string code)...

0 references
public static List<BillDAL> GetByUserId(int userId)...

0 references
public static List<BillDAL> FindBySQL(string sql)...

0 references
public static bool UpdateStatus(int id, string status)...
```

Kiểm soát trạng thái hóa đơn

```
//Bill Status Constants
public const string StatusWaiting = "waiting";
public const string StatusFinish = "finish";
public const string StatusCancel = "cancel";

private static bool IsValidStatus(string? status)
{
    if (string.IsNullOrWhiteSpace(status)) return false;
    var s = status.Trim().ToLowerInvariant();
    return s == StatusWaiting || s == StatusFinish || s == StatusCancel;
}

private static string NormalizeStatus(string? status)
{
    if (string.IsNullOrWhiteSpace(status)) return StatusWaiting;
    var s = status.Trim().ToLowerInvariant();
    return IsValidStatus(s) ? s : StatusWaiting;
}
```

Xây dựng phương thức ***tạo hoá đơn cùng nhiều sản phẩm***

```
public static int InsertWithDetails(BillDAL bill, List<BillDetailDAL>? details)
{
    if (bill == null) throw new ArgumentNullException(nameof(bill));
    var billId = Insert(bill);
    if (billId <= 0) return 0;
    if (details == null || details.Count == 0) return billId;
    var insertedDetailIds = new List<int>();
    try
    {
        foreach (var d in details)
        {
            if (d == null) continue;
            d.BillId = billId;
            if (d.Quantity <= 0) throw new ArgumentException("Detail quantity must be > 0", nameof(details));
            if (d.Price < 0) throw new ArgumentException("Detail price must be >= 0", nameof(details));
            var detailId = BillDetailDAL.Insert(d);
            if (detailId <= 0)
            {
                throw new InvalidOperationException("Failed to insert bill detail");
            }
            insertedDetailIds.Add(detailId);
        }
        return billId;
    }
    catch
    {
        foreach (var id in insertedDetailIds)
        {
            try { BillDetailDAL.Delete(id); } catch {  }
        }
        try { Delete(billId); } catch {  }
        return 0;
    }
}
```

❑ Tự động kiểm tra lại tổng tiền hóa đơn khi cập nhật (**Update**)

```
public static bool Update(BillDAL bill)
{
    if (bill == null) throw new ArgumentNullException(nameof(bill));
    var status = NormalizeStatus(bill.Status);
    try
    {
        var details = BillDetailDAL.GetByBillId(bill.Id);
        long computedTotal = 0;
        foreach (var d in details)
        {
            if (d.Quantity <= 0) return false;
            if (d.Price < 0) return false;
            computedTotal += (long)d.Price * d.Quantity;
        }
        bill.Total = computedTotal > int.MaxValue ? int.MaxValue : (int)computedTotal;
    }
    catch { return false; }
    string sql = "UPDATE bills SET user_id = @1, code = @2, created_at = @3, total = @4, status = @5 WHERE id = @6; SELECT @@ROWCOUNT;";
    var parameters = new List<object>
    {
        bill.UserId,
        bill.Code ?? string.Empty,
        bill.CreatedAt,
        bill.Total,
        status,
        bill.Id
    };
    var res = DBUtil.executeScalar(sql, parameters);
    if (res != null && Convert.ToInt32(res) > 0) { bill.Status = status; return true; }
    return false;
}
```

- Xây dựng phương thức ***cập nhật trạng thái***
- Trạng thái đơn hàng chỉ được cập nhật với điều kiện sau:
  - Từ waiting -> finish
  - Từ waiting -> cancel
  - Từ finish -> cancel (**Chỉ Manager**)

```
public static bool UpdateStatus(int id, string status)
{
    if (string.IsNullOrWhiteSpace(status)) throw new ArgumentNullException(nameof(status));
    var newStatus = NormalizeStatus(status);
    var existing = GetById(id);
    if (existing == null) return false;
    var currentStatus = existing.Status?.Trim().ToLowerInvariant();
    if (!IsValidStatus(currentStatus)) return false;
    if (string.Equals(currentStatus, newStatus, StringComparison.OrdinalIgnoreCase)) return false;

    var allowed =
        (currentStatus == StatusWaiting && (newStatus == StatusFinish || newStatus == StatusCancel))
     || (currentStatus == StatusFinish && newStatus == StatusCancel);

    if (!allowed) return false;
    string sql = "UPDATE bills SET status = @1 WHERE id = @2; SELECT @@ROWCOUNT;";
    var res = DBUtil.executeScalar(sql, new List<object> { newStatus, id });
    return res != null && Convert.ToInt32(res) > 0;
}
```

- Xây dựng phương thức ***lấy danh sách đơn hàng theo nhân viên***

```
public static List<BillDAL> GetByUserId(int userId)
{
    var list = new List<BillDAL>();
    string sql = "SELECT id, user_id, code, created_at, total, status FROM bills
WHERE user_id = @1";
    var dt = DBUtil.queryDataTable(sql, new List<object> { userId });
    foreach (DataRow row in dt.Rows)
        list.Add(FromDataRow(row));
    return list;
}
```

CHI TIẾT ĐƠN HÀNG

Cơ sở dữ liệu

dbo.bill_details		
int	id	PK
int	bill_id	FK
int	drink_id	FK
int	quantity	
int	price	



Entity Class

```
public class BillDetailDAL
{
    4 references
    public int Id { get; set; }
    4 references
    public int BillId { get; set; }
    3 references
    public int DrinkId { get; set; }
    6 references
    public int Quantity { get; set; }
    6 references
    public int Price { get; set; }
```

BillDetailDAL

- Các phương thức ***Business Logic*** ở ***BillDetailDAL***

```
2 references
public static List<BillDetailDAL> GetByBillId(int billId)...

0 references
public static List<BillDetailDAL> GetByDrinkId(int drinkId)...

0 references
public static int AddDrinkToBill(int billId, int drinkId, int quantity, int price)...

0 references
public static bool UpdateQuantity(int detailId, int newQuantity)...
```

BillDetailDAL

- Xây dựng phương thức ***thêm thức uống vào chi tiết hoá đơn***
  - Chỉ có thể thêm khi đơn hàng đang ở trạng thái waiting
  - Nếu đã có thức uống ở hoá đơn thì tăng số lượng lên 1
  - Nếu chưa có thức uống sẽ thêm mới vào hoá đơn

BillDetailDAL

```
public static int AddDrinkToBill(int billId, int drinkId, int quantity, int price)
{
    var bill = BillDAL.GetById(billId);
    if (bill == null) return 0;
    try
    {
        var details = GetByBillId(billId);
        var existing = details.Find(d => d.DrinkId == drinkId);
        if (existing != null)
        {
            existing.Quantity += quantity;
            existing.Price = price;
            var ok = Update(existing);
            if (!ok) return 0;
            BillDAL.Update(bill);
            return existing.Id;
        }
        else
        {
            var detail = new BillDetailDAL
            {
                BillId = billId,
                DrinkId = drinkId,
                Quantity = quantity,
                Price = price
            };
            var newId = Insert(detail);
            if (newId <= 0) return 0;
            BillDAL.Update(bill);
            return newId;
        }
    }
    catch { return 0; }
}
```

BillDetailDAL

- Xây dựng phương thức ***cập nhật số lượng của đồ uống trong hoá đơn***
  - Chỉ có thể cập nhật số lượng khi đơn hàng đang ở trạng thái waiting
  - Nếu số lượng đồ uống <= 0 thì xoá đồ uống khỏi đơn hàng
  - Sau khi cập nhật số lượng sẽ thực hiện cập nhật lại tổng giá tiền đơn hàng

```
public static bool UpdateQuantity(int detailId, int newQuantity)
{
    if (detailId <= 0) throw new ArgumentOutOfRangeException(nameof(detailId));
    if (newQuantity <= 0) throw new ArgumentOutOfRangeException(nameof(newQuantity));
    var detail = GetById(detailId);
    if (detail == null) return false;
    detail.Quantity = newQuantity;
    var ok = Update(detail);
    if (!ok) return false;
    var bill = BillDAL.GetById(detail.BillId);
    if (bill == null) return false;

    return BillDAL.Update(bill);
}
```

Cài đặt ***PosController***

- GET /index: Trang giao diện POS bán hàng
- POST /init: Khởi tạo đơn hàng mới
- POST /update-quantity: Cập nhật số lượng thức uống
- POST /checkout: Hoàn tất và thanh toán đơn hàng
- POST /cancel: Huỷ đơn hàng đang tạo

Giao diện bán hàng

Danh sách sản phẩm

Tìm kiếm...



Cà phê sữa 45.000đ

Thêm



Trà đào 50.000đ

Thêm



Bạc xỉu 40.000đ

Thêm



Sinh tố bơ 55.000đ

Thêm



Matcha Latte 60.000đ

Thêm

Đơn hàng

	Tên	SL	Thành tiền	
	Cà phê sữa	- 2 +	90.000đ	X
	Trà đào	- 1 +	50.000đ	X
	Sinh tố bơ	- 1 +	55.000đ	X

Tạm tính 195.000đ

Tổng **195.000đ**

Hoàn thành đơn | Huỷ đơn

PosController

- Cài đặt mã cho phương thức ***GET /index***
  - Hiển thị danh sách thức uống được kích hoạt
  - Nếu có billId ở URL hiển thị danh sách sản phẩm đã thêm vào đơn hàng

```
[Authorize(Policy = "EmployeeOrManager")]
public IActionResult Index(int? billId)
{
    var drinks = DrinkDAL.GetAll().Where(d => d.Active).ToList();
    if (billId.HasValue && billId.Value > 0)
    {
        var bill = BillDAL.GetById(billId.Value);
        if (bill != null)
        {
            var details = BillDetailDAL.GetByBillId(bill.Id);
            ViewBag.InitialBill = bill;
            ViewBag.InitialDetails = details;
            ViewBag.CurrentBillId = bill.Id;
        }
    }
    return View(drinks);
}
```

PosController

- Nghiệp vụ yêu cầu phương thức ***POST /init.*** Phương thức này được gọi khi nhân viên nhấn nút "Thêm" tại danh sách thức uống trên giao diện POS
  - Nếu chưa tồn tại đơn hàng đang mở
    - Khởi tạo đơn hàng mới
    - Thêm thức uống với số lượng là 1
  - Nếu đã tồn tại đơn hàng đang mở
    - Kiểm tra thức uống có tồn tại trong đơn hàng không
      - ✓ Nếu chưa có thêm mới với số lượng là 1
      - ✓ Nếu đã có tăng số lượng thức uống lên 1

POST /init

```
if (drinkId <= 0) return BadRequest(new { success = false, error = "Invalid drinkId." });
int userId = 0;
var cid = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
if (!string.IsNullOrWhiteSpace(cid) && int.TryParse(cid, out var uid))
    userId = uid;
var drink = DrinkDAL.GetById(drinkId);
if (drink == null || !drink.Active) return NotFound(new { success = false, error = "Drink not found or inactive." });
BillDAL? bill = null;
if (userId > 0)
{
    var userBills = BillDAL.GetByUserId(userId);
    bill = userBills.FirstOrDefault(b => string.Equals(b.Status?.Trim(), BillDAL.StatusWaiting, StringComparison.OrdinalIgnoreCase));
}
var createdNewBill = false;
if (bill == null)
{
    var newBill = new BillDAL
    {
        UserId = userId,
        Code = DateTime.UtcNow.ToString("yyyyMMddHHmmss") + "_" + Guid.NewGuid().ToString("N").Substring(0, 6),
        CreatedAt = DateTime.Now,
        Total = 0,
        Status = BillDAL.StatusWaiting
    };
    var newBillId = BillDAL.Insert(newBill);
    if (newBillId <= 0) return BadRequest(new { success = false, error = "Cannot create bill." });
    bill = BillDAL.GetById(newBillId);
    createdNewBill = true;
}
if (bill == null) return BadRequest(new { success = false, error = "Cannot determine bill." });
var detailId = BillDetailDAL.AddDrinkToBill(bill.Id, drinkId, 1, drink.Price);
if (detailId <= 0)
{
    if (createdNewBill) { try { BillDAL.Delete(bill.Id); } catch { } }
    return BadRequest(new { success = false, error = "Failed to add product to bill." });
}
var updatedBill = BillDAL.GetById(bill.Id)!;
var updatedDetails = BillDetailDAL.GetByBillId(bill.Id);
return Ok(new { success = true, bill = updatedBill, details = updatedDetails });
```

PosController

- Nghiệp vụ yêu cầu phương thức ***POST /update-quantity.*** Phương thức này được gọi khi nhân viên nhấn nút "Tăng (+)", "Giảm (-)" và "Xoá (x)" ở đơn hàng thức uống
- Chỉ được thực hiện khi đơn hàng đang ở trạng thái "waiting"
- Nghiệp vụ từng yêu cầu
  - Khi nhấn vào nút "Tăng (+)" sẽ thực hiện tăng số lượng của thức uống đang chọn lên 1
  - Khi nhấn vào nút "Giảm (-)" sẽ thực hiện giảm số lượng của thức uống đi 1
    - Nếu sau khi giảm số lượng thức uống là 0 sẽ thực hiện xoá sản phẩm đó khỏi đơn hang
    - Ngược lại sẽ giảm số lượng thức uống đi 1
  - Khi ấn vào nút "Xoá (x)" sẽ thực hiện xoá sản phẩm khỏi đơn hàng

POST /update-quantity

```
try
{
    var bill = BillDAL.GetById(billId);
    if (bill == null) return BadRequest(new { success = false, error = "Bill not found." });
    if (!string.Equals(bill.Status?.Trim(), BillDAL.StatusWaiting, StringComparison.OrdinalIgnoreCase))
        return BadRequest(new { success = false, error = "Bill not editable." });
    var existing = BillDetailDAL.GetByBillId(billId).FirstOrDefault(d => d.DrinkId == drinkId);
    var act = (action ?? string.Empty).Trim().ToLowerInvariant();

    //code tiện ích hỗ trợ xử lý

    if (act == "incr") {
        if (existing != null) UpdateExisting(existing.Quantity + 1);
        else Add(1);
    }
    else if (act == "decr") {
        if (existing != null) {
            if (existing.Quantity <= 1) DeleteExisting();
            else UpdateExisting(existing.Quantity - 1);
        }
    }
    else if (act == "delete") { DeleteExisting(); }
    else if (quantity.HasValue) {
        if (quantity.Value <= 0) DeleteExisting();
        else if (existing != null) UpdateExisting(quantity.Value);
        else Add(quantity.Value);
    }
    else {
        if (existing != null) UpdateExisting(existing.Quantity + 1);
        else Add(1);
    }
    BillDAL.Update(bill);
    var updatedBill = BillDAL.GetById(billId)!;
    var updatedDetails = BillDetailDAL.GetByBillId(billId);
    return Ok(new { success = true, bill = updatedBill, details = updatedDetails });
}
catch (Exception ex) { return BadRequest(new { success = false, error = ex.Message }); }
```

POST /update-quantity

```
//code tiện ích hỗ trợ xử lý
void Add(int qty)
{
    var drink = DrinkDAL.GetById(drinkId);
    BillDetailDAL.AddDrinkToBill(billId, drinkId, qty, drink?.Price ?? 0);
}

void UpdateExisting(int qty)
{
    if (existing == null) return;
    existing.Quantity = qty;
    BillDetailDAL.Update(existing);
}

void DeleteExisting()
{
    if (existing != null) BillDetailDAL.Delete(existing.Id);
}
```

PosController

- Xây dựng mã cho phương thức ***POST /checkout*** và ***POST /cancel***.

```
public IActionResult Checkout(int billId)
{
    if (billId <= 0) return BadRequest(new { success = false, error = "Invalid billId." });
    var bill = BillDAL.GetById(billId);
    if (bill == null) return NotFound(new { success = false, error = "Bill not found." });
    var okUpdate = BillDAL.Update(bill);
    if (!okUpdate) return BadRequest(new { success = false, error = "Failed to update bill total." });
    var ok = BillDAL.UpdateStatus(billId, BillDAL.StatusFinish);
    if (!ok) return BadRequest(new { success = false, error = "Cannot set bill to finish." });
    var finished = BillDAL.GetById(billId);
    return Ok(new { success = true, bill = finished });
}

public IActionResult Cancel(int billId)
{
    if (billId <= 0) return BadRequest(new { success = false, error = "Invalid billId." });

    var bill = BillDAL.GetById(billId);
    if (bill == null) return NotFound(new { success = false, error = "Bill not found." });

    var ok = BillDAL.UpdateStatus(billId, BillDAL.StatusCancel);
    if (!ok) return BadRequest(new { success = false, error = "Cannot cancel bill." });

    return Ok(new { success = true });
}
```



XÂY DỰNG CHỨC NĂNG BÁO CÁO THỐNG KÊ

THỐNG KÊ SẢN PHẨM BÁN CHẠY

- Nghiệp vụ yêu cầu ***Procedure thống kê 5 sản phẩm bán chạy nhất***
  - Chỉ lấy hoá đơn đã hoàn tất **(status == "finish")**
  - Sắp xếp theo tổng số lượng đã bán
  - Hiển thị thông tin chi tiết thức uống kèm theo tổng doanh thu từng thức uống
  - Có thể lọc theo khoản thời gian (tuỳ chọn)

THỐNG KÊ SẢN PHẨM BÁN CHẠY

- Xây dựng mã cho ***Procedure thống kê 5 sản phẩm bán chạy nhất***

```
CREATE PROCEDURE sp_top_5_best_selling_drinks
    @from_date DATE = NULL,
    @to_date   DATE = NULL
AS
BEGIN
    SET NOCOUNT ON;

    SELECT TOP 5
        d.id   AS drink_id,
        d.name AS drink_name,
        SUM(bd.quantity) AS total_quantity_sold,
        SUM(bd.quantity * bd.price) AS total_revenue
    FROM bill_details bd
    INNER JOIN drinks d ON bd.drink_id = d.id
    INNER JOIN bills b ON bd.bill_id = b.id
    WHERE
        b.status = 'finish'
        AND (@from_date IS NULL OR b.created_at >= @from_date)
        AND (@to_date   IS NULL OR b.created_at <= @to_date)
    GROUP BY d.id, d.name
    ORDER BY total_quantity_sold DESC;
END
GO
```

THỐNG KÊ DOANH THU

- Nghiệp vụ yêu cầu ***Procedure thống kê doanh thu***
  - Chỉ lấy hoá đơn đã hoàn tất **(status == "finish")**
  - Thống kê doanh thu theo từng ngày trong hệ thống POS
  - Doanh thu được tính dựa trên tổng giá trị hoá đơn
  - Hiển thị tổng số hoá đơn và tổng doanh thu theo từng ngày
  - Sắp xếp theo thời gian tăng dần
  - Có thể lọc theo khoản thời gian (tuỳ chọn)

THỐNG KÊ DOANH THU

- Xây dựng mã cho ***Procedure thống kê doanh thu***

```
CREATE PROCEDURE sp_revenue_by_day
    @from_date DATE,
    @to_date   DATE
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        created_at        AS revenue_date,
        COUNT(id)         AS total_bills,
        SUM(total)        AS total_revenue
    FROM bills
    WHERE
        status = 'finish'
        AND created_at BETWEEN @from_date AND @to_date
    GROUP BY created_at
    ORDER BY revenue_date;
END
GO
```

THỐNG KÊ ENTITY

```
public class StatisticDAL
{
    1 reference
    public class BestSellingDrink
    {
        0 references
        public int DrinkId { get; set; }
        0 references
        public string? DrinkName { get; set; }
        0 references
        public int TotalQuantitySold { get; set; }
        0 references
        public int TotalRevenue { get; set; }

        0 references
        public BestSellingDrink() { }
    }

    1 reference
    public class RevenueByDay
    {
        0 references
        public DateTime RevenueDate { get; set; }
        0 references
        public int TotalBills { get; set; }
        0 references
        public int TotalRevenue { get; set; }

        0 references
        public RevenueByDay() { }
    }
}
```

- Thức uống bán chạy Entity
- Doanh thu Entity

❑ Xây dựng mã cho phương thức ***getTop5BestSellingDrinks***

```
public static List<BestSellingDrink> getTop5BestSellingDrinks(DateTime fromDate, DateTime toDate)
{
    var result = new List<BestSellingDrink>();
    string sql = "EXEC sp_top_5_best_selling_drinks @1, @2";
    var dt = DBUtil.queryDataTable(sql, new List<object> { fromDate, toDate });

    foreach (DataRow row in dt.Rows)
    {
        result.Add(new BestSellingDrink
        {
            DrinkId = row.Field<int>("drink_id"),
            DrinkName = row.Field<string?>("drink_name"),
            TotalQuantitySold = row.Field<int>("total_quantity_sold"),
            TotalRevenue = row.Field<int>("total_revenue")
        });
    }

    return result;
}
```

❑ Xây dựng mã cho phương thức ***getRevenueByDay***

```
public static List<RevenueByDay> getRevenueByDay(DateTime fromDate, DateTime toDate)
{
    var result = new List<RevenueByDay>();
    string sql = "EXEC sp_revenue_by_day @1, @2";
    var dt = DBUtil.queryDataTable(sql, new List<object> { fromDate, toDate });

    foreach (DataRow row in dt.Rows)
    {
        result.Add(new RevenueByDay
        {
            RevenueDate = row.Field<DateTime>("revenue_date"),
            TotalBills = row.Table.Columns.Contains("total_bills") && row["total_bills"] != DBNull.Value
                         ? Convert.ToInt32(row["total_bills"]) : 0,
            TotalRevenue = row.Table.Columns.Contains("total_revenue") && row["total_revenue"] != DBNull.Value
                           ? Convert.ToInt32(Convert.ToDecimal(row["total_revenue"])) : 0
        });
    }

    return result;
}
```

Summary

- Xây dựng chức năng quản lý hoá đơn
- Xây dựng chức năng báo cáo thống kê
  - Thống kê sản phẩm bán chạy
  - Thống kê doanh thu




FPT Education
FPT POLYTECHNIC
Thank you